

7. Selection Sort on Random, Reverse and Sorted Arrays

Aim

To implement **Selection Sort** on random, reverse, and already sorted arrays and analyze its time complexity.

Algorithm

1. Start with the first element of the array.
2. Find the smallest element in the unsorted part.
3. Swap it with the first unsorted element.
4. Repeat until the complete array is sorted.
5. Perform the same process for random, reverse, and sorted arrays.

Procedure

1. Create three arrays: random, reverse, and sorted.
2. Apply Selection Sort to each array.
3. Find the minimum element in each pass.
4. Swap the minimum element with the current position.
5. Display the sorted arrays.
6. Compare the results.

main.py	Visualize	Run ▶	Output
<pre>1 def selection_sort(a): 2 n = len(a) 3 4 for i in range(n - 1): 5 min_index = i 6 7 for j in range(i + 1, n): 8 if a[j] < a[min_index]: 9 min_index = j 10 11 a[i], a[min_index] = a[min_index], a[i] 12 13 return a 14 random_array = [64, 25, 12, 22, 11] 15 reverse_array = [50, 40, 30, 20, 10] 16 sorted_array = [10, 20, 30, 40, 50] 17 18 print("Random Array:", selection_sort(random_array)) 19 print("Reverse Array:", selection_sort(reverse_array)) 20 print("Sorted Array:", selection_sort(sorted_array))</pre>			<pre>Random Array: [11, 12, 22, 25, 64] Reverse Array: [10, 20, 30, 40, 50] Sorted Array: [10, 20, 30, 40, 50]</pre>

Result

Thus, Selection Sort was successfully implemented for random, reverse, and sorted arrays. The time complexity was found to be **$O(n^2)$** for all three cases.